

Exp No:5	Principal Component Analysis
Date:	

AIM:

To implement **Principal Component Analysis (PCA)** on a given dataset in order to reduce dimensionality, identify correlated variables, and determine the minimum number of principal components that retain maximum information (variance).

ALGORITHM:

1. Collect a dataset with **multiple numerical attributes**.
2. Arrange the dataset in matrix form.
3. Compute the **mean** of each attribute.
4. Normalize the dataset by subtracting the mean from each value.
5. Compute the **covariance matrix** of the normalized data.
6. Calculate the **eigenvalues and eigenvectors** of the covariance matrix.
7. Sort the eigenvalues in **descending order**.
8. Select the top **k eigenvectors** corresponding to the largest eigenvalues.
9. Project the original data onto the selected principal components.
10. Analyze variance retained and interpret the reduced data.

PROGRAM:

Principal Component Analysis (PCA) Implementation

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

Step 1: Create Dataset

```
data = {
    'Internal': [45, 50, 48, 60, 55],
    'Assignment': [30, 28, 35, 40, 38],
    'Attendance': [200, 220, 210, 250, 240],
    'FinalExam': [3.5, 3.7, 3.6, 3.9, 3.8]
}
```

```
df = pd.DataFrame(data)
print("Original Dataset:\n", df)
```

Step 2: Standardize the Data

```
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)
```

```
print("\nStandardized Data:\n", scaled_data)
```

OUTPUT:

Original Dataset:

	Internal	Assignment	Attendance	FinalExam
0	45	30	98	85
1	50	28	99	87
2	48	35	100	96
3	60	40	76	69
4	55	38	89	88

Standardized Data:

```
[[-1.24197138 -0.91738926 0.61601725 0.
[-0.30108397 -1.35424129 0.72602033 0.22645541]
[-0.67743894 0.17474081 0.83602341 1.24550474]
[ 1.58069085 1.26687088 -1.80405051 -1.81164325]
[ 0.63980344 0.83001885 -0.37401047 0.33968311]]]
```

Principal Components:

	Principal Component 1	Principal Component 2
0	1.413624	-0.716918
1	1.283680	-0.737221
2	1.306140	1.004912
3	-3.235389	-0.402168
4	-0.768056	0.851395

Explained Variance Ratio:

```
[0.8204911 0.14769635]
```

Total Variance Retained:

```
0.9681874453703446
```

```
PS C:\Users\kavim>
```

```

# Step 3: Apply PCA
pca = PCA(n_components=2)
principal_components = pca.fit_transform(scaled_data)

# Step 4: Create DataFrame for Principal Components
pca_df = pd.DataFrame(
    data=principal_components,
    columns=['Principal Component 1', 'Principal Component 2']
)

print("\nPrincipal Components:\n", pca_df)

# Step 5: Display Explained Variance
print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)

print("\nTotal Variance Retained:")
print(np.sum(pca.explained_variance_ratio_))

```

Preparation	20	
Implementation	30	
Viva	15	
Record	10	
Total	75	

RESULT:

- The dataset was successfully transformed using **Principal Component Analysis**.
- Dimensionality was reduced from **4 attributes to 2 principal components**.
- The selected components retained **maximum variance**, minimizing information loss.
- PCA helped in identifying **correlated variables** and simplifying data analysis.
- The experiment demonstrates that PCA is effective for **feature reduction and data compression**.

GitHub Link : <https://github.com/KavimugilRajasekar/MachineLearningLab.git>